

Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

D5 – Test Case Specification and Test Results

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor Dr. Omaia Al-Omari

Semester & Academic Year Senior Project I & II – Academic Year 2026–2027

Submission Date _____

Supervisor's Signature _____

Approval Date _____

Table of Contents

Table of Contents i
List of Tables iii
Test Case Specification and Test Results 1
3.5 Testing & Validation 1
3.5.1 Test strategy 1
3.5.2 Test environment, tools and responsibilities 4
3.5.3 Entry and exit criteria 4
3.5.4 Continuous integration and the evidence pipeline 5
3.5.5 Metrics 6
3.5.6 Test inventory 7
3.5.7 User acceptance testing 8
B.1 Test environment and evidence 9
B.2 Requirements traceability matrix 10
B.3 Test case specification 18
B.3.1 Policy language and engine 18
B.3.2 Gateway and protocol adapters 20
B.3.3 Taint tracking 22
B.3.4 Detector and result handling 24
B.3.5 Approvals and audit 25
B.3.6 Administration, user interface, demonstration, benchmark and storage 28
B.4 Security test cases 31
B.5 Test results 33
B.5.1 Execution summary 33
B.5.2 Code coverage 33
B.5.3 Static analysis and dependency scanning 34
B.5.4 Performance 34
B.5.5 Benchmark results on benign tasks 36
B.5.6 Audit verification and decision replay 38
B.5.7 LLM smoke run 39

B.5.8 Results pending team-supplied data 39
B.6 Known limitations and open items 40
References 42

List of Tables

Table 1: Test levels and the artefacts they verify (V-model) 2
Table 2: Exit criteria and their status for the reported version 4
Table 3: Continuous-integration jobs 5
Table 4: Automated tests by area 7
Table 5: Planned user acceptance scenarios 8
Table 6: Evidence files behind this appendix 9
Table 7: Requirements traceability matrix 10
Table 8: Verification status summary 17
Table 9: Test cases TC-01 to TC-08, policy language and engine 18
Table 10: Test cases TC-09 to TC-17, gateway and protocol adapters 20
Table 11: Test cases TC-18 to TC-23, taint tracking 22
Table 12: Test cases TC-24 to TC-30, detector and result handling 24
Table 13: Test cases TC-31 to TC-39, approvals and audit 25
Table 14: Test cases TC-40 to TC-46, administration, UI, demonstration, benchmark and storage 28
Table 15: Security test cases ST-01 to ST-23 31
Table 16: Security tests by STRIDE category 32
Table 17: Test execution summary 33
Table 18: Statement coverage by package 33
Table 19: Static analysis and dependency scan results 34
Table 20: Added latency per tool call (ms) 34
Table 21: Benign benchmark results per defence and approver 36
Table 22: Audit verification and replay results 38
Table 23: Results that need team-supplied attack data 39

Test Case Specification and Test Results

This document is deliverable D5 of the Mirsad senior project. It describes the test strategy and presents the test case specification, the requirements traceability matrix and the test results.

This document is an extract of the Mirsad Final Report, where its content appears as Section 3.5 and Appendix B. Section, figure and table numbers, and references to other sections, chapters and appendices, follow that report.

3.5 Testing & Validation

This section explains how Mirsad is verified, meaning that it is built correctly against its specification, and validated, meaning that it does its job of stopping hijacked agents without getting in the way of legitimate work. It covers the test strategy, the test levels and tools, the entry and exit criteria, the continuous-integration pipeline, the metrics, the test inventory and the planned user acceptance testing. The test case specification, the requirements traceability matrix (Table 7) and the full test report are in Appendix B. Section 4.2 discusses the results.

3.5.1 Test strategy

The strategy uses the concepts and documentation structure of ISO/IEC/IEEE 29119 [1] and is organised as a V-model. Each development artefact on the left of the V has a test level on the right that checks it (Table 1). The requirements in Section 3.2 are checked by end-to-end tests and by the benchmark. The architecture and the engineering contract (`docs/dev/BUILD_SPEC.md`) are checked by integration and API tests, and the design of each package by unit tests. The test design followed three principles.

1. **Requirement-driven and risk-based.** Every functional and non-functional requirement is traced to at least one automated test (Table 7). Most effort went to the parts whose failure would either let an attack through or block legitimate work: the policy engine, the taint rules, the fail-closed paths of the gateway, the approval workflow and the audit chain.
2. **Negative cases are first-class.** Every rule has a case where it must fire and a case where it must not. The bulk-export rule is tested at 50 and 51 recipients (boundary values). Each constraint operator is tested with one passing and one violating value (equivalence classes). The legitimate counterpart of an "untrusted destination" is a reply to the sender of an e-mail, and that reply must not be blocked. The security suite attacks the gateway itself with forged tokens, database tampering and path traversal.
3. **Deterministic and offline.** Tests use temporary SQLite files, seeded synthetic data (the fictional Najd Trading Co. on reserved `.example` domains), explicit timestamps

for rate-limit windows, and fake components where a unit has to be isolated. Tests that need the local language model carry the pytest marker `llm` and are left out of the default run, because model output is not stable enough to serve as a pass/fail gate.

Test data and attack material. This report contains no prompt-injection text, and apart from the eight Demo Console scenarios (`demo/scenarios.py`), which the demo tests load to check that they plant their text into the simulated workspace (Section 3.6.4), neither does the repository. A test that needs content to be scored as an injection uses a neutral placeholder marker together with a stub detector rule. Everything that happens after the detector produces its score, that is thresholds, sanitising, spotlighting, taint registration, audit and replay, then runs the production code. These tests therefore show how Mirsad *handles* a detection. They do not show how *well* the detector detects real attacks. Detector accuracy and attack success rate are measured separately, once the team's attack-labelled data is complete (Section 3.5.6).

Table 1: Test levels and the artefacts they verify (V-model)

Level	Verifies	Scope and techniques	Tools and location
Unit	Design of each package (policy, taint, detector, audit, approvals, auth, adapters, demo, bench)	One package at a time, with fakes for its neighbours. Equivalence classes and boundary values for constraints and rate limits, decision tables for reason combination, concurrency tests for the rate limiter, audit chain and approvals	pytest, pytest-asyncio; `tests/<area>/`
Integration	Engineering contract between packages	Real policy engine, taint tracker, detector, sanitiser, audit log, approvals and in-process MCP server wired together; the same storage, audit, approval and authentication code against PostgreSQL	`tests/integration/`, `tests/api/test_integration_real.py`, `tests/taint/test_taint_integration.py`
API and contract	REST, MCP and OpenAI-compatible interfaces	Application driven through an HTTP client: status codes, roles, schemas, error bodies; OpenAPI document compared with the specified routes; web client parsed against captured server records	httpx; `tests/api/`, `tests/adapters/`, `web/tests/contract.test.tsx`
End-to-end	User-visible behaviour	Real CLI (`mirsad init --demo`), the application built by `mirsad serve` on a real port, plain	`tests/api/test_e2e_http.py`, `tests/adapters/test_adapters_mcp_proxy.py`

Level	Verifies	Scope and techniques	Tools and location
		HTTP calls, an approval sent from a second operating-system process, the stdio MCP proxy started as a subprocess	
UI	Dashboard and approver PWA	Component tests in a simulated browser (jsdom): approve/deny flow, Arabic right-to-left rendering, bilingual dictionaries, SSE parsing, colour contrast of the design tokens	Vitest, Testing Library; `web/tests/`
Security	Security requirements from the STRIDE analysis	23 security tests (ST-01 to ST-23; 45 test cases) against the running application, grouped by STRIDE category [2]	`tests/security/test_gateway_security.py`
Static analysis	Code quality and insecure code patterns	Linting with security rules, static application security testing, strict TypeScript type checking	Ruff, Bandit, TypeScript compiler
Dependency scanning	Known vulnerabilities in third-party packages	Advisory databases for Python and npm packages	pip-audit, npm audit
Performance	NFR1 and NFR11	1,000 benign HTTP calls, sequential and with 50 concurrent sessions; server in the same process and in a separate process	`bench/latency.py`
System evaluation	Objectives 1–3 (Section 1.3)	Mirsad-Bench: 48 benign tasks in English and Arabic, five defence configurations, two approver behaviours, audit verification and replay of every decision; LLM smoke run; attack episodes once the team's attack files exist	`bench/`, `scripts/run_bench.sh`
Acceptance (UAT)	Stakeholder needs: use cases UC-01, UC-05, UC-08, UC-13 to UC-15 and NFR4	Scenario-based sessions in which the supervisor and two to three security practitioners operate the system on prepared requests in the simulated workspace (Section 3.5.7)	Planned; questionnaire in Appendix F, Section D

3.5.2 Test environment, tools and responsibilities

All recorded runs were made on a team member's laptop (Apple silicon, macOS) with Python 3.12 and Node.js. The CI workflow is configured to run the same suites on Linux; no CI run is recorded in `reports/`. Python tests use pytest with pytest-asyncio and pytest-cov. HTTP-level tests drive the FastAPI application through httpx, either in memory or over a real socket served by uvicorn. The PostgreSQL tests start a throwaway server from the `pgserver` package, so no database has to be installed. Web tests use Vitest with Testing Library and jsdom. The `llm`-marked tests and the LLM smoke run use Qwen2.5-7B [3] served locally by Ollama.

Each workstream owner writes and maintains the unit and API tests for their own packages (Section 3.4.3). The security suite, Mirsad-Bench and the latency benchmark belong to the security-testing and benchmark workstream (Abdulmalik bin Saedan). Under the agreed working rules, every pull request needs a second reviewer and a passing CI run before it is merged (Section 3.4.4).

3.5.3 Entry and exit criteria

A test level starts only when its entry criteria hold:

- **Unit and API tests:** the public interface of the component is fixed in the engineering contract and the code passes the linter.
- **Integration and end-to-end tests:** the unit suites of every package involved pass.
- **Benchmark runs:** the scenario files validate (`python -m bench.loader`) and the full automated suite passes, so that benchmark numbers always come from tested code.
- **Reported results:** the evidence pipeline (Section 3.5.4) has been run on the version being reported.

Table 2 lists the exit criteria for a release candidate, together with the values measured for the version described in this report. The first ten criteria are gates: a candidate that misses any of them is not released. The lint and type-check gate is enforced by the CI workflow, but its result is not stored with the evidence, so it is shown as not recorded. The last two are project success criteria that cannot be measured until attack data exists. They are reported as open rather than estimated.

Table 2: Exit criteria and their status for the reported version

Criterion	Target	Measured	Status
Automated Python tests (all areas, `llm` marker excluded)	all pass, none skipped	1,281 of 1,281 passed, 0 skipped	Met
Web UI tests	all pass	194 of 194 passed	Met
Statement coverage of `src/mirsad` (NFR9)	≥ 80%	94.4%	Met

Criterion	Target	Measured	Status
Lint (Ruff) and type check (`tsc --noEmit`)	no findings	no report stored in `reports/` (B.5.3)	Not recorded
Static application security testing (Bandit)	no medium or high findings	0 findings of any severity	Met
Known-vulnerable dependencies	none	pip-audit: 0 of 124 packages; npm audit: 0	Met
STRIDE security suite	all pass	45 test cases passed	Met
Median added latency per tool call (NFR1)	≤ 200 ms, judged on the sequential median and on the separate-process layout	2.7 ms sequential; 166 ms with 50 sessions (separate process)	Met
Benign false-positive rate with full protection	≤ 5%	0.0%	Met (rules-only detector)
Benchmark decisions that replay to the same action (FR6.3)	100%	272 of 272 and 272 of 272	Met
Detector F1 on the English / Arabic held-out test sets	≥ 0.85	[pending] / [pending]	Pending data
Attack success rate reduction against the unprotected baseline	≥ 80%	[pending]	Pending data

3.5.4 Continuous integration and the evidence pipeline

The CI workflow (`.github/workflows/ci.yml`) is configured to run on every pull request and every push to the main branch. Table 3 shows its three jobs. A failing step stops the job, and under the agreed working rules a pull request is not merged with a red build. No CI run is recorded in `reports/`, so the results in this report come from the local evidence pipeline described below.

Table 3: Continuous-integration jobs

Job	Steps	The build fails when
python	Install the project with development extras; `ruff check` on source, tests, demo, bench and scripts; `pytest -m "not llm"` with coverage; `bandit -r src`; `pip-audit`	any lint finding, failing test, Bandit finding or vulnerable dependency
web	`npm ci`; `tsc --noEmit`; Vitest; production build with Vite	a type error, failing test or build error
docker	Build the container image (runs only after both other jobs pass)	the image does not build

Two checks are not yet enforced by CI. The 80% coverage target is read from the coverage report rather than enforced with a `--cov-fail-under` threshold, and `npm audit` runs in the evidence pipeline and in `make security` but not in the workflow.

The numbers in this report come from a separate **evidence pipeline**, `scripts/run_evidence.sh`. It re-runs the Python suite (writing JUnit XML and a coverage JSON file), the web suite, Bandit, pip-audit and npm audit, both latency benchmarks and both scripted Mirsad-Bench runs (through `scripts/run_bench.sh`), and it trains the detector when labelled data is present. `scripts/collect_results.py` then gathers every result into `reports/results_summary.json`. The report builder (`scripts/build_docs.py`) reads each number in this document from that file when the document is generated. A number in the text therefore cannot drift away from the evidence, and a value that has not been produced yet prints as "[pending]" instead of being estimated.

3.5.5 Metrics

The following metrics are collected. Definitions of the benchmark metrics follow `docs/benchmark.md`.

- **Test execution:** number of tests per area, and passed, failed and skipped counts. A test is one pytest item or one Vitest case, so each parametrised case counts separately.
- **Code coverage:** statement coverage of the `mirsad` package. Branch coverage is not collected.
- **Static analysis and dependencies:** number of findings per tool, and number of packages with a known advisory.
- **Latency:** *added latency* per call, which is the HTTP round trip through Mirsad minus the direct in-process tool call, paired call by call. It is reported as p50, p95 and p99, together with throughput and HTTP errors. Time spent waiting for a human is excluded by design.
- **Benchmark:** *utility*, the share of episodes whose success predicate holds. *Benign false-positive rate (FPR)*, the share of benign calls denied automatically. *Approval friction*, the share of benign calls handed to a human. *Final block rate*, the share of benign calls that did not run in the end. *Result false-positive rate*, the share of benign tool results that the detector sanitised or blocked. *Approval recall*, the share of tasks that legitimately need approval in which a human was asked. *Gateway latency*, the decision plus result inspection per call. *Attack success rate (ASR)*, and *ASR reduction* = 1 − ASR(defence) / ASR(none).
- **Audit integrity:** the result of hash-chain verification and the share of decisions whose replay gives the same action.
- **Detector:** precision, recall, F1, false-positive rate and ROC-AUC on each held-out split, and scan latency for a 2 KB text.

- **Usability (NFR4):** an analytical estimate of approval time with the Keystroke-Level Model [4] (Section 4.2.8). Approval times of real users have not been measured yet.
- **Acceptance:** for each acceptance scenario of Section 3.5.7, whether each participant accepts it, with the number of participants.

FPR and friction are reported separately on purpose. A policy can deny no benign call and still cost a lot of human time if it escalates too often, and the two numbers show these two costs.

3.5.6 Test inventory

Table 4 lists the automated tests by area. The Python suite has 56 test files. Two further files hold only `llm`-marked tests (`tests/demo/test_llm.py` and `tests/bench/test_bench_llm.py`) and are not part of the recorded run.

Table 4: Automated tests by area

Area	Files	What the tests cover	Owner	Tests
policy	7	Schema and validator messages, every constraint operator, rate limiter, rules, sink resolution, ReDoS guard for policy regexes, shipped policies and policy documentation	Bader Alshuraie	265
adapters	4	MCP proxy (stdio and HTTP), OpenAI-compatible proxy, upstream manager, Python SDK	Bader Alshuraie	78
api	10	Gateway pipeline, Guard API, admin API, authentication and roles, HTTP hardening, SSE stream, CLI, demo endpoints, real-component integration and end-to-end HTTP	Bader Alshuraie	185
taint	4	Value extraction, the four taint rules and their exemptions, session tracker, replay from snapshots	Bader Abdullah Abbod Alyamini	93
audit	8	Hash chain and tamper detection, query and export, redaction, storage, approval service, approver summaries, notifier and Web Push, authentication core	Bader Abdullah Abbod Alyamini	240
detector	9	Normalisation, language identification, rules, model persistence, ensemble, sanitiser and spotlighting, dataset builder, training and metrics	Abdullah Naif Abdullah Al-Orabi	227
demo	4	Simulated workspace, MCP workspace server, demo scenarios, demo agent loop	Abdulmalik bin Saedan	72
bench	7	Scenario loader and schema, episodes and predicates, attack-episode construction, planting, survival markers and stop attribution (neutral placeholder text only), defence configurations, agents, metrics, runner and latency benchmark	Abdulmalik bin Saedan	68
security	1	STRIDE security suite ST-01 to ST-23	Abdulmalik bin Saedan	45
integration	2	PostgreSQL compatibility, cross-module wiring	All members	8
Python total	**56**			**1,281**
web	10	API client, SSE client, contract with server records,	Mohammed	194

Area	Files	What the tests cover	Owner	Tests
		approver PWA and dashboard pages, i18n dictionaries, colour contrast, countdown, span highlighting	Yasser Alanssari	

Two measurements cannot be produced by this inventory yet: detector precision, recall and F1, and the attack success rate. Both depend on attack-labelled data written by the team (Section 3.6.4). Detector metrics are produced by running `make detector` after the team adds labelled batches and separate held-out test batches to `data/detector/raw/` (procedure in `docs/detector.md`, Section 11.4). The attack success rate is produced by Mirsad-Bench, which already executes attack episodes (Section 4.1.12), once the team adds injection tasks and attack templates to `bench/scenarios/attacks/`. Until then, every place in this report that needs these values shows "[pending]".

3.5.7 User acceptance testing

The levels above verify the system against its specification. User acceptance testing validates it with the people it is built for: whether an approver, an administrator and an auditor can complete their main tasks with the real interfaces and accept the result. Table 5 lists the planned acceptance scenarios. They are run in the simulated Najd Trading Co. workspace with prepared, benign requests, by the supervisor and two to three security practitioners recruited by the team, each of whom records "accept" or "reject" per scenario with a reason. Section D of the questionnaire in Appendix F is completed after the session. No session has been held yet; outcomes will be reported with the number of participants, and no result is estimated before then.

Table 5: Planned user acceptance scenarios

ID	Use case / requirement	Actor	Steps	Acceptance criterion	Evidence
AT-01	UC-01, BR4	Agent developer	Run the demonstration agent through the gateway on a benign task that e-mails an outside supplier, then on one that e-mails a colleague	The outside e-mail is paused and not sent until a decision is made; the internal e-mail is sent without a pause	Audit events of both sessions
AT-02	UC-05, NFR4	Approver	Receive a paused request on a phone, read the summary and approve or deny it	The participant can say why the action was paused from the summary alone and decides within 30 seconds	Request and decision times from the approval record; questionnaire items D1 and D2
AT-03	UC-05, NFR4	Approver	As AT-02 with the interface in Arabic	As AT-02; the participant judges the Arabic layout and summary readable	As AT-02
AT-	UC-08	Administrator	Edit the policy to allow	Errors are reported with	`policy.activated`

ID	Use case / requirement	Actor	Steps	Acceptance criterion	Evidence
04			a new partner domain, validate it, review the difference, activate it, then roll back	line numbers; the change applies to the next call without a restart; rollback restores the earlier version	audit events
AT-05	UC-13, UC-14	Auditor	Find the events of one session, verify the hash chain and export the events as JSON	The events are found with the filters, verification reports an intact chain and the export verifies offline	Export file and verification output
AT-06	UC-15	Auditor	Replay a stored decision from the dashboard	The replay reports the same action and the policy version that made the original decision	Replay output

This appendix is the test case specification and test report (deliverable D5). Section 3.5 describes the test strategy. This appendix contains the evidence: the requirements traceability matrix (B.2), the specification of representative test cases in the style of ISO/IEC/IEEE 29119-3 [1] (B.3), the security test cases (B.4), the measured results (B.5) and the open items (B.6). Every number below is read from the result files listed in Table 6 when the document is built. A value shown as "[pending]" has not been produced yet. Python test files are under `tests/<area>/` and web test files under `web/tests/`. A test name such as `test_policy_engine.py::test_unknown_agent_is_denied` identifies one automated test function.

B.1 Test environment and evidence

The recorded results were collected into `reports/results_summary.json` on 2026-10-14T07:58:56+00:00 (UTC). The tests, scans and latency benchmarks were run by `scripts/run_evidence.sh` (the separate-process latency run finished at 2026-10-14T06:51:13+00:00), on macOS-26.5.1-arm64-arm-64bit with Python 3.12.15. The Mirsad-Bench runs were executed from scratch by `scripts/run_bench.sh` and the LLM smoke command at 2026-10-14T07:31:54+00:00 (first audit check of the denying run), with the current default policy, in which CRM results are scanned. The injection detector ran in its rules-only fallback mode, because no trained model is packaged yet (detector mode recorded by the latency benchmark: rules_only). Every detector-dependent number in this appendix was therefore measured without the statistical model.

Table 6: Evidence files behind this appendix

File	Content
`reports/tests/junit.xml`	Python test run (`pytest -m "not llm"`), one record per test with its outcome

File	Content
`reports/tests/vitest.json`	Web test run (Vitest), one record per test case
`reports/coverage.json`	Statement coverage of the `mirsad` package (coverage.py)
`reports/security/bandit.json`, `pip-audit.json`, `npm-audit.json`	Static analysis and dependency scan reports
`reports/bench/latency.json`, `latency-process.json`	Latency benchmark with the server in the same process and in a separate process
`reports/bench/scripted-benign-deny/`, `scripted-benign-oracle/`, `llm-smoke/`	Mirsad-Bench runs: episode records, summaries, audit-verification and replay results, figures
`reports/results_summary.json`	All of the above collected by `scripts/collect_results.py`; the source of the placeholders in this report

B.2 Requirements traceability matrix

Table 7 maps every functional requirement, non-functional requirement and business rule of Section 3.2 to the design component that implements it, the automated tests that verify it, and its verification status. The status values are:

- **Verified:** the acceptance criterion is checked by passing automated tests or by recorded measurements.
- **Partially verified:** part of the criterion is checked automatically. The reason names the part that is not.
- **Pending data:** the criterion needs attack-labelled data that the team is still preparing (Section 3.5.6).

TC and ST numbers refer to the test case specifications in B.3 and B.4.

Table 7: Requirements traceability matrix

ID	Design component	Verifying tests	Status

FR 1.1	MCP proxy (`adapters/mcp_proxy.py`), upstream manager (`adapters/upstreams.py`)	`test_adapters_mcp_proxy.py::test_tools_are_filtered_by_policy_and_schemas_pass_through`
		`test_adapters_mcp_proxy.py::test_allowed_call_returns_inspected_content`
		`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
		`test_integration_real.py::test_mcp_streamable_http_endpoint_on_the_real_app`
		`test_adapters_upstreams.py::test_stdio_upstream_lists_calls_and_reconnects_after_crash`
			Verified (TC-09 to TC-11)

FR 1.2	OpenAI-compatible proxy (`adapters/openai_proxy.py`)	`test_adapters_openai_proxy.py::test_denied_tool_call_is_removed_and_explained`
		`test_adapters_openai_proxy.py::test_tool_messages_are_inspected_before_forwarding`
		`test_adapters_openai_proxy.py::test_partial_denial_keeps_allowed_calls`
		`test_adapters_openai_proxy.py::test_request_validation`
			Verified (TC-12, TC-13)

ID	Design component	Verifying tests	Status

FR 1.3	Guard API (`api/routers/guard.py`), Python SDK (`sdk/client.py`)	`test_api_guard.py::test_tool_call_allow`
		`test_api_guard.py::test_tool_call_deny`
		`test_api_guard.py::test_invoke_denied_tool_is_never_executed`
		`test_adapters_sdk.py::test_guard_denied_call_does_not_run`
		`test_adapters_sdk.py::test_guard_blocked_result_raises`
			Verified (TC-14, TC-15)

FR 1.4	`PolicyEngine.visible_tools`; tool listing in the MCP proxy, OpenAI proxy and `GET /v1/tools`	`test_api_guard.py::test_visible_tools_are_filtered_by_policy`
		`test_adapters_openai_proxy.py::test_tool_definitions_are_filtered_by_policy`
		`test_adapters_mcp_proxy.py::test_visible_tools_failure_exposes_nothing`
		`test_policy_engine.py::test_unknown_agent_profile_and_visible_tools`
			Verified (TC-09)

FR 2.1	Policy schema and validator (`policy/schema.py`), engine (`policy/engine.py`), monitor mode in `gateway.py`	`test_policy_schema.py::test_every_shipped_policy_validates`
		`test_policy_schema.py::test_yaml_syntax_error_has_line_number`
		`test_policy_schema.py::test_unknown_key_reports_line_and_suggestion`
		`test_policy_engine.py::test_tool_resolution_precedence`
		`test_gateway.py::test_monitor_mode_allows_but_audits_would_be_decision`
			Verified (TC-01, TC-02, TC-16)

FR 2.2	Constraint evaluator (`policy/constraints.py`)	`test_policy_constraints.py::test_operator_passes_and_violates` (19 operator cases)
		`test_policy_constraints.py::test_on_violation_require_approval_is_propagated`
		`test_policy_constraints.py::test_domain_in_accepts_subdomains_but_not_lookalikes`
		`test_policy_constraints.py::test_not_pattern_sees_through_fullwidth_and_zero_width_obfuscation`
			Verified (TC-03, TC-04)

FR 2.3	Sliding-window rate limiter (`policy/ratelimit.py`)	`test_policy_engine.py::test_rate_limit_window_on_export_customers`
		`test_policy_ratelimit.py::test_hit_counts_calls_inside_the_window`
		`test_policy_ratelimit.py::test_window_slides_and_old_calls_expire`
		`test_policy_ratelimit.py::test_concurrent_hits_are_all_counted`
			Verified (TC-05)

FR 2.4	Deny-overrides combination (`combine_reasons` in `models.py`), bilingual reason messages	`test_policy_engine.py::test_constraint_violations_combine_with_deny_overrides`
		`test_policy_constraints.py::test_arabic_text_in_messages_is_present_for_every_operator`
		`test_gateway.py::test_explain_decision_is_bilingual`
		`test_taint_rules.py::test_reasons_are_bilingual_serialisable_and_name_the_tool`
			Verified (TC-06)

FR 2.5	Policy store and activation (`api/routers/policies.py`, `Gateway.set_policy`), `mirsad policy activate`	`test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`
		`test_api_admin.py::test_invalid_policy_is_rejected_and_not_activated`
		`test_gateway.py::test_policy_hot_swap_applies_to_next_call`
		`test_cli.py::test_policy_activate_persists_and_audits`
		`ui.test.tsx`: "rolls back to an older version after confirmation"
			Verified (TC-40)

FR	Tool profiles	`test_e2e_http.py::test_agent_tool_calls_follow_the_def`	Partially

ID	Design component	Verifying tests	Status

3.1	(`PolicyEngine.tool_profile`), `Gateway.inspect_result`	`ault_policy`
		`test_gateway.py::test_benign_untrusted_result_passes_with_spotlight`
		`test_gateway.py::test_result_of_disallowed_tool_is_treated_as_untrusted`
		`test_taint_tracker.py::test_register_result_updates_sources_sensitivity_injection_and_values`
			verified: trust and sensitivity labels on returned results are asserted; the inspection audit event stores the full tool profile (`gateway.py`), but no test asserts that field

FR 3.2	Value extraction (`taint/extract.py`), tracker (`taint/tracker.py`)	`test_taint_extract.py::test_structured_results_keep_field_paths_and_trusted_flags`
		`test_taint_extract.py::test_argument_candidates_find_values_embedded_in_long_and_encoded_strings`
		`test_taint_extract.py::test_iban_spacing_and_case_normalised_and_lookalikes_rejected`
		`test_taint_integration.py::test_structured_result_plus_free_text_content_records_both`
			Verified (TC-23)

FR 3.3	Taint rules (`taint/rules.py`) combined in `Gateway.check_call`	`test_taint_rules.py::test_untrusted_content_then_external_sink_requires_approval`
		`test_taint_rules.py::test_confidential_crm_phone_inside_external_email_body_is_flagged`
		`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
		`test_taint_rules.py::test_replying_to_the_sender_is_not_flagged_as_untrusted_destination`
		`test_taint_rules.py::test_crm_email_is_not_an_untrusted_destination_even_if_also_in_untrusted_text`
		`test_api_guard.py::test_untrusted_result_then_external_call_needs_approval`
			Verified (TC-18 to TC-21)

FR 3.4	`TAINT_AFTER_INJECTION` rule; per-session state in `taint/tracker.py`	`test_taint_rules.py::test_high_risk_tool_after_injection_requires_approval`
		`test_taint_rules.py::test_low_risk_tool_after_injection_is_not_flagged`
		`test_taint_rules.py::test_high_risk_tool_without_injection_is_not_flagged`
		`test_taint_tracker.py::test_sessions_are_isolated_by_session_and_agent`
		`test_gateway.py::test_injection_is_sanitized_and_taints_session`
			Verified (TC-22, TC-29)

FR 4.1	Detector (`detector/normalize.py`, `rules.py`, `model.py`, `detector.py`, `language.py`); training and evaluation (`train.py`, `build_dataset.py`)	`test_detector.py::test_result_fields_and_latency`
		`test_detector.py::test_spans_point_into_the_original_text`
		`test_detector.py::test_benign_arabic_business_text_is_below_flag_threshold`
		all tests in `test_language.py`
		`test_train.py::test_metrics_report_structure`
		`test_train.py::test_tuning_never_looks_at_test_splits`
			Pending data: score, label, language and spans are verified; F1 ≥ 0.85 needs the team's attack-labelled test sets (B.5.8)

ID	Design component	Verifying tests	Status

FR 4.2	Normaliser and evasion signals (`detector/normalize.py`, `detector/rules.py`)	`test_normalize.py::test_zero_width_characters_are_removed_and_signalled`
		`test_normalize.py::test_cyrillic_lookalikes_inside_latin_word_are_folded`
		`test_normalize.py::test_stacked_diacritics_raise_tashkeel_signal`
		`test_normalize.py::test_bidi_controls_are_removed_and_signalled`
		`test_detector.py::test_spans_survive_invisible_characters`
		`test_rules.py::test_evasion_signals_amplify_only_real_hits`
			Partially verified: normalisation and obfuscation signals are verified; the detection rate on obfuscated attack texts needs team-supplied data

FR 4.3	Sanitiser and spotlighting (`detector/sanitize.py`), `Gateway.inspect_result`	`test_sanitize.py::test_exact_rules_remove_only_the_span`
		`test_sanitize.py::test_block_mode_withholds_everything_with_bilingual_notice`
		`test_sanitize.py::test_spotlight_structure`
		`test_sanitize.py::test_positive_detection_without_spans_fails_closed`
		`test_gateway.py::test_suspicious_result_is_flagged_not_modified`
		`test_api_guard.py::test_invoke_sanitizes_poisoned_upstream_output`
			Verified with neutral placeholder content (TC-27 to TC-29)

FR 5.1	Approval workflow (`approvals/service.py`), `Gateway.check_call`	`test_e2e_http.py::test_external_email_waits_for_an_approval_sent_from_a_second_process`
		`test_gateway.py::test_approval_granted_by_human_allows_and_excludes_wait_from_latency`
		`test_policy_engine.py::test_approval_always_tool`
		`test_api_guard.py::test_tool_call_approved_by_human_via_api`
			Verified (TC-31)

FR 5.2	Approver summaries (`approvals/summary.py`), notifier with SSE and Web Push (`approvals/notifier.py`), approver PWA	`test_approval_summary.py::test_english_summary_matches_the_reference_example`
		`test_approval_summary.py::test_arabic_summary_is_arabic_and_mentions_tool_and_destination`
		`test_approval_summary.py::test_every_reason_code_yields_a_short_summary`
		`test_approval_service.py::test_request_triggers_web_push_and_prunes_stale_subscriptions`
		`test_api_stream.py::test_sse_relays_approval_lifecycle_and_audit_events`
		`contract.test.tsx`: "yields the \"data came from\" source and the reasons that paused the call"
			Verified (TC-32, TC-33)

FR 5.3	`ApprovalService` decide, wait and expiry; approver PWA (`web/src/approver/`)	`test_approval_service.py::test_wait_timeout_marks_expired_and_publishes`
		`test_approval_service.py::test_decide_twice_raises_value_error`
		`test_approval_service.py::test_concurrent_deciders_exactly_one_wins`
		`test_gateway.py::test_approval_timeout_denies_as_expired`
		`ui.test.tsx`: "lists the pending request, explains it and approves it after confirmation"
			Verified (TC-34, TC-35, TC-42, ST-09)

FR 6.1	Hash-chained audit log (`audit/log.py`), redaction (`audit/redact.p`	`test_audit_chain.py::test_verify_detects_modified_payload`
		`test_audit_chain.py::test_verify_detects_deleted_row`
		`test_audit_chain.py::test_verify_detects_reordered_rows`
			Verified (TC-36, TC-37, ST-06, ST-07)

ID	Design component	Verifying tests	Status
	y)	`test_audit_chain.py::test_verify_detects_forged_rechain_without_key` `test_audit_chain.py::test_payload_is_redacted_before_storage`	

FR 6.2	`AuditLog.query` and `export`, audit router (`api/routers/audit.py`), dashboard Activity page	`test_audit_query.py::test_query_filters_by_each_column`
		`test_audit_query.py::test_query_time_range`
		`test_audit_query.py::test_query_text_search_payload_tool_agent_case_insensitive`
		`test_audit_query.py::test_query_pagination_and_ordering`
		`test_audit_query.py::test_export_csv_rows_and_header`
		`test_api_admin.py::test_audit_list_filter_and_detail`
		`test_api_admin.py::test_audit_export_json_and_csv`
			Verified (TC-38)

FR 6.3	`Gateway.replay`, taint snapshots, versioned policy store	`test_gateway.py::test_replay_decision_matches_original`
		`test_gateway.py::test_replay_uses_the_policy_version_recorded_in_the_event`
		`test_gateway.py::test_replay_reproduces_taint_from_stored_snapshot`
		`test_api_admin.py::test_replay_after_policy_change_uses_recorded_version`
		`test_e2e_http.py::test_audit_verify_replay_and_csv_export`
		benchmark replay sweep (Table 22)
			Verified (TC-39)

FR 7.1	Authentication core (`auth.py`), agent, user and tool routers	`test_api_admin.py::test_agent_crud_shows_key_once`
		`test_api_admin.py::test_user_crud_never_returns_hashes`
		`test_api_admin.py::test_last_admin_is_protected`
		`test_api_admin.py::test_tools_listing_shows_profiles_per_agent`
		`test_auth_core.py::test_rotate_agent_key_invalidates_old_key`
		`ui.test.tsx`: "creates a user with a server-generated password and shows it once"
			Verified (TC-41, ST-04, ST-12)

FR 7.2	Statistics and stream routers (`api/routers/stats.py`, `stream.py`), detector playground, dashboard Overview	`test_api_admin.py::test_stats_overview_shape_and_counts`
		`test_api_admin.py::test_stats_timeseries_buckets`
		`test_api_stream.py::test_sse_relays_approval_lifecycle_and_audit_events`
		`test_api_guard.py::test_detector_playground`
		`sse.test.ts` (SSE parser and `connectStream`)
			Partially verified: statistics, the SSE stream and the browser SSE client are tested; no automated UI test shows the Overview page updating on a live event

FR 8.1	Simulated workspace (`demo/world.py`), MCP workspace server, demo agent and scenarios, `/api/demo/*`	`test_world.py::test_data_is_fake` and the other tests in `test_world.py`
		`test_server.py::test_list_tools`
		`test_scenarios.py::test_apply_scenario_plants_payload`
		`test_api_demo.py::test_unprotected_agent_leaks_to_attacker_inbox`
		`test_api_demo.py::test_protected_agent_is_stopped_by_the_gateway`
			Verified with a scripted model stub (TC-43); the live-model test (`tests/demo/test_llm.py`) is `llm`-marked and not in the recorded run

FR	Mirsad-Bench	`test_bench_loader.py::test_suite_has_enough_benign_tas`	Partially

ID	Design component	Verifying tests	Status

8.2	(`bench/`)	`ks_in_both_languages`
		`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`
		`test_bench_metrics.py::test_fpr_friction_and_utility`
		`test_bench_metrics.py::test_attack_success_rate_math`
		`test_bench_run.py::test_runner_writes_outputs_and_resumes`
		`test_bench_attacks.py::test_policy_denial_stops_a_large_export_under_full_defense`
		`test_bench_attacks.py::test_approval_and_taint_attribution`
		`test_bench_attacks.py::test_runner_auto_mode_runs_both_kinds_and_writes_the_asr_figure`
			verified (TC-44, TC-46): `scripts/run_bench.sh` regenerates the benign result tables (B.5.5) and executes attack episodes when attack files exist; the attack tables need team-supplied attack files

NF R1	Gateway pipeline; latency benchmark (`bench/latency.py`)	`test_bench_run.py::test_latency_benchmark_small` latency benchmark over 1,000 calls (Table 20)	Verified

NF R2	Protocol adapters; demo agent over the OpenAI-compatible API to a local model	`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
		`test_integration_real.py::test_mcp_streamable_http_endpoint_on_the_real_app`
		`test_adapters_openai_proxy.py::test_allowed_tool_calls_pass_through_unchanged`
		LLM smoke run with Qwen2.5-7B (B.5.7)
			Verified

NF R3	`auth.py`, HTTP hardening (`api/security.py`), redaction, regex guard (`policy/regex_guard.py`)	Security suite ST-01 to ST-23 (B.4)
		all tests in `test_auth_core.py` and `test_redaction.py`
		`test_api_http.py::test_security_headers_on_every_response`
		`test_api_http.py::test_request_body_over_1mb_is_rejected`
		Bandit, pip-audit and npm audit (Table 19)
			Verified

NF R4	Approver PWA (`web/src/approver/`), approver summaries	`ui.test.tsx`: approver PWA tests all tests in `i18n.test.ts` `test_approval_summary.py::test_every_reason_code_yields_a_short_summary`	Partially verified: Arabic right-to-left rendering and the approve flow are tested; the 30-second decision time rests on an analytical Keystroke-Level Model estimate (Section 4.2.8), and no approver has been timed yet
NF R5	`Dockerfile`, `docker-compose.yml`,	`test_e2e_http.py::test_init_demo_then_login_and_overview` `test_cli.py::test_init_demo_creates_agent_and_honours_admin_password`	Partially verified: the local start with

ID	Design component	Verifying tests	Status
	`mirsad init` and `mirsad serve`, `scripts/run_demo.sh`	CI `docker` job (defined; no run recorded)	`init` and `serve` is tested end to end; a `docker compose up` run is not recorded in `reports/`

NFR6	Fail-closed paths in `gateway.py` and the adapters; approval expiry	`test_gateway.py::test_approval_timeout_denies_as_expired`
		`test_gateway.py::test_execute_executor_exception_returns_safe_error_and_is_audited`
		`test_gateway.py::test_execute_times_out_slow_tools`
		`test_gateway.py::test_audit_failure_fails_closed`
		`test_gateway.py::test_policy_exception_fails_closed`
		`test_gateway.py::test_detector_failure_blocks_untrusted_result`
		`test_adapters_openai_proxy.py::test_gateway_failures_fail_closed`
			Verified (TC-15, TC-17, TC-34, ST-13)

NFR7	Audit event schema (`gateway.py`, `audit/log.py`)	`test_gateway.py::test_principal_is_recorded_in_audit_payload`
		`test_gateway.py::test_allow_low_risk_call_is_audited_with_replay_payload`
		`test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`
			Verified (ST-09)

NFR8	Redaction (`audit/redact.py`); synthetic demo data	`test_redaction.py::test_saudi_national_id_and_iqama_masked_to_last_four`
		`test_redaction.py::test_iban_masked_except_country_and_last_four`
		`test_redaction.py::test_luhn_valid_card_masked_to_last_four`
		`test_world.py::test_data_is_fake`
			Verified (ST-11)

NFR9	Whole code base	Statement coverage (Table 18)
		Ruff (Table 19)
		`test_e2e_http.py::test_openapi_document_covers_the_spec_routes`
		`test_policy_docs.py::test_documentation_exists_and_covers_every_operator`
			Verified

NFR10	Web UI (`web/src/`), design tokens	all tests in `contrast.test.ts` (53 colour pairs in both themes: text ≥ 4.5:1, focus ring and UI components ≥ 3:1) all tests in `i18n.test.ts` `ui.test.tsx` (controls located by accessible role and name)	Partially verified: contrast and accessible names are tested automatically; keyboard navigation, no colour-only signals and reduced motion depend on the UI review checklist, which is not recorded in `reports/`

ID	Design component	Verifying tests	Status

NFR11	Single-node gateway; storage on SQLite and PostgreSQL (`storage/database.py`)	latency benchmark with 50 concurrent sessions (Table 20)
		all tests in `test_postgres.py`
		`test_storage_db.py::test_schema_compiles_for_postgresql`
		`test_gateway.py::test_concurrent_calls_keep_a_consistent_audit_chain`
			Verified (TC-45); tail latency under concurrency is high (B.5.4)

BR1	Default deny in `PolicyEngine`	`test_policy_engine.py::test_unknown_agent_is_denied`
		`test_policy_engine.py::test_unmatched_tool_with_default_deny`
		`test_gateway.py::test_unlisted_tool_is_denied`
			Verified (TC-08, ST-22)

BR2	`approval: always` for `transfer_funds` and `delete_customer` in the default policy	`test_policy_engine.py::test_approval_always_tool`
		`test_e2e_http.py::test_agent_tool_calls_follow_the_default_policy`
		`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`
			Verified (TC-06, TC-44)

BR3	`count` and `limit` constraints; rule `no-bulk-export-to-email`	`test_policy_engine.py::test_default_bulk_export_rule`
		`test_policy_engine.py::test_default_rule_regex_is_linear_on_adversarial_bodies`
		`test_e2e_http.py::test_agent_tool_calls_follow_the_default_policy`
			Verified (TC-07)

BR4	`domain_in` constraint on `send_email`; `TAINT_UNTRUSTED_DESTINATION`	`test_policy_engine.py::test_external_recipient_needs_approval_by_constraint`
		`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
		`test_policy_scenarios.py::test_scenario_injected_email_exfiltration`
			Verified (TC-04, TC-20)

BR5	`defaults.approval_timeout_s`; expiry in `ApprovalService`	`test_approval_service.py::test_wait_timeout_marks_expired_and_publishes`
		`test_approval_service.py::test_record_deadline_expires_without_explicit_timeout`
		`test_gateway.py::test_approval_timeout_denies_as_expired`
			Verified (TC-34); tests use short timeouts, the 120 s default is configuration

BR6	Audit router exposes only list, get, verify, replay and export	Route definitions in `api/routers/audit.py` ST-06 and ST-07 (direct database edits are detected)	Partially verified: no write route exists, but no test asserts their absence

Table 8: Verification status summary

Requirement group	Verified	Partially verified	Pending data	Total
Functional requirements (FR1.1–FR8.2)	21	4	1	26
Non-functional requirements (NFR1–NFR11)	8	3	0	11
Business rules (BR1–BR6)	5	1	0	6
Total	**34**	**8**	**1**	**43**

B.3 Test case specification

The 46 test cases below are a representative selection from the automated suites, covering every area and every requirement group. Each one is specified with the fields of an ISO/IEC/IEEE 29119-3 test case: identifier, requirement, objective, preconditions and inputs, expected result, actual result and the automated procedure that executes it. All test cases are automated. The actual result is taken from the recorded run (`reports/tests/junit.xml` and `reports/tests/vitest.json`), in which every test passed. Where a test needs content that the detector scores as an injection, it uses a neutral placeholder rather than real attack text (Section 3.5.1).

B.3.1 Policy language and engine

Table 9: Test cases TC-01 to TC-08, policy language and engine

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-01	FR 2.1	Every shipped policy is valid	Each YAML file in `configs/policies/`	`validate_policy` returns no errors; each policy defines at least one agent	Pass	`test_policy_schema.py::test_every_shipped_policy_validates`
TC-02	FR 2.1	Validation errors are located and actionable	Policy with a misspelt tool key (`sinkk`)	Error starts with "line 8, column 44" and suggests `sink`	Pass	`test_policy_schema.py::test_unknown_key_reports_line_and_suggestion`
TC-03	FR 2.2	Every constraint operator accepts a valid value and rejects an invalid one	19 operator cases, each with a passing and a violating value (e.g. `max: 10` with 10 and 11)	Passing value: no reason. Violating value: one `CONSTRAINT_VIOLATION` (deny) with evidence `param`, `operator`, `expected`, `actual` and different English and Arabic messages	Pass	`test_policy_constraints.py::test_operator_passes_and_violates`
T	FR	domai	domain_in	Subdomain	Pas	test_policy_constraints.py::test_domain_in_ac

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
C-04	2.2, BR 4	`n_in` is not fooled by look-alike domains	`: [najd-trading.example]`; an address at a subdomain and addresses at two look-alike domains	accepted; both look-alikes violate `domain_in`	s	`cepts_subdomains_but_not_lookalikes`
TC-05	FR 2.3	Sliding-window rate limit	`export_customers` limited to 3 calls per 3,600 s; calls at t = 0, 1, 2, 3 s and at t = 3,603 s	Calls 1–3 allowed; call 4 denied with `RATE_LIMIT` (count 4, max 3); allowed again once the window has slid	Pass	`test_policy_engine.py::test_rate_limit_window_on_export_customers`
TC-06	FR 2.4, BR 2	Deny overrides require-approval	`transfer_funds` with `amount_sar` 20,000 (limit 10,000; tool always needs approval)	Reasons `TOOL_RISK`, `CONSTRAINT_VIOLATION`, `APPROVAL_ALWAYS`; combined action DENY	Pass	`test_policy_engine.py::test_constraint_violations_combine_with_deny_overrides`
TC-07	BR 3	Bulk-export rule at its boundary	`send_email` whose body lists 51 addresses, then 50	51: `RULE_MATCH` by `no-bulk-export-to-email`, deny. 50: rule does not fire	Pass	`test_policy_engine.py::test_default_bulk_export_rule`
TC-08	BR 1	Unknown agents are denied	Call from an agent id that is not in the policy	Single reason `UNKNOWN_AGENT`, action DENY; English and Arabic messages name the agent	Pass	`test_policy_engine.py::test_unknown_agent_is_denied`

B.3.2 Gateway and protocol adapters

Table 10: Test cases TC-09 to TC-17, gateway and protocol adapters

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-09	FR 1.1, FR 1.4	MCP clients see only permitted tools, with unchanged schemas	MCP client connected to the proxy; the policy makes two of the upstream tools visible	`tools/list` returns exactly those two tools; input schemas equal the upstream schemas	Pass	`test_adapters_mcp_proxy.py::test_tools_are_filtered_by_policy_and_schemas_pass_through`
TC-10	FR 1.1	A denied MCP call is not executed and is explained	Gateway denies `send_email` to an outside address	MCP error result; the upstream tool never runs; the text names the tool, the reason code and the data source in English and Arabic	Pass	`test_adapters_mcp_proxy.py::test_denied_call_is_not_executed_and_explained`
TC-11	FR 1.1, NFR2	The stdio proxy works as a separate process	An MCP client launches `run_stdio_proxy` as a subprocess that forwards to a live HTTP gateway	Tool list filtered; allowed call returns sanitised content; denied call returns an error; only the allowed tool runs upstream	Pass	`test_adapters_mcp_proxy.py::test_run_stdio_proxy_end_to_end_subprocess`
TC-12	FR 1.2	Denied tool calls are removed	The upstream model returns a `send_email` tool	`tool_calls` removed; `finish_reason` becomes	Pass	`test_adapters_openai_proxy.py::test_denied_tool_call_is_removed_and_explained`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		from the model's reply	call that the gateway denies	`stop`; appended assistant text explains the block in English and Arabic		
TC-13	FR 1.2	Tool messages are inspected before they reach the model	Conversation whose `tool` message carries flagged content	Forwarded message contains the sanitise marker instead of the flagged text; tool name resolved from the preceding `tool_calls`	Pass	`test_adapters_openai_proxy.py::test_tool_messages_are_inspected_before_forwarding`
TC-14	FR 1.3	The SDK decorator blocks denied calls	Function decorated with `client.guard`; the gateway denies the call	`MirsadBlocked` raised at stage `call`; the function body does not run	Pass	`test_adapters_sdk.py::test_guard_denied_call_does_not_run`
TC-15	FR 1.3, NFR6	The SDK fails closed	Gateway unreachable (connection refused); guarded `delete_customer`	`MirsadError` raised; the guarded function does not run	Pass	`test_adapters_sdk.py::test_guard_fails_closed_when_gateway_unreachable`
TC-16	FR 2.1	Monitor mode records but does not enforce	Policy in `mode: monitor`; a call to a disallowed tool, then to a tool that always needs	Both calls return ALLOW; the audit event stores the would-be action `deny` with `enforced`	Pass	`test_gateway.py::test_monitor_mode_allows_but_audits_would_be_decision`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			approval	`: false`; no approval is requested		
TC-17	NFR6	Tool failures neither crash the gateway nor leak internals	Executor raises an exception whose message contains an internal file path	Generic error returned to the agent without the path; `tool_call.error` audit event stored	Pass	`test_gateway.py::test_execute_executor_exception_returns_safe_error_and_is_audited`

B.3.3 Taint tracking

Table 11: Test cases TC-18 to TC-23, taint tracking

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-18	FR 3.3	Untrusted content followed by an external sink is escalated	Session reads an e-mail (untrusted), then sends a message to an outside partner	`TAINT_UNTRUSTED_TO_EXTERNAL`, REQUIRE_APPROVAL, risk 0.6; evidence names the source tool and call id	Pass	`test_taint_rules.py::test_untrusted_content_then_external_sink_requires_approval`
TC-19	FR 3.3	A confidential value inside free text is caught	Session reads a CRM customer, then e-mails an outside address with the customer's phone number	`TAINT_CONFIDENTIAL_TO_EXTERNAL` (kind phone, parameter `body`, field `phone`, source `get_customer`), REQUIRE_APPROVAL	Pass	`test_taint_rules.py::test_confidential_crm_phone_inside_external_email_body_is_flagged`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			inside the body			
TC-20	FR 3.3, BR 4	A destination chosen by untrusted content is denied	Inbox listing whose message body contains an outside address; the agent then sends to that address	`TAINT_UNTRUSTED_DESTINATION`, DENY, risk 0.9, field `[0].body`; English and Arabic messages name the domain	Pass	`test_taint_rules.py::test_attacker_address_inside_an_email_body_controls_destination`
TC-21	FR 3.3	A reply to the sender is not an untrusted destination	Session reads an e-mail and replies to its sender (trusted `from` field)	No `TAINT_UNTRUSTED_DESTINATION`; `TAINT_UNTRUSTED_TO_EXTERNAL` still present because the sink is external	Pass	`test_taint_rules.py::test_replying_to_the_sender_is_not_flagged_as_untrusted_destination`
TC-22	FR 3.4	High-risk tools need approval after a detection	A session result was labelled `injection`; the agent then calls `delete_customer` (critical)	`TAINT_AFTER_INJECTION`, REQUIRE_APPROVAL, risk 0.8; the source call id is recorded	Pass	`test_taint_rules.py::test_high_risk_tool_after_injection_requires_approval`
TC-23	FR 3.2	Arabic text matching ignores diacritics	Arabic CRM note recorded from a confidential result; an outgoing e-mail	`TAINT_CONFIDENTIAL_TO_EXTERNAL` with kind `ngram`, field `notes` and readable Arabic evidence	Pass	`test_taint_rules.py::test_arabic_confidential_note_copied_with_different_diacritics_is_flagged`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			body copies the note with different diacritics			

B.3.4 Detector and result handling

Table 12: Test cases TC-24 to TC-30, detector and result handling

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-24	FR 4.2	Zero-width characters are removed and reported	Ordinary words with a zero-width space, joiner or word joiner inserted	Normalised text without them; `zero_width_chars` signal; every offset maps back to the original character	Pass	`test_normalize.py::test_zero_width_characters_are_removed_and_signalled`
TC-25	FR 4.2	Homoglyphs are folded	Latin words containing Cyrillic look-alike letters	Folded to Latin; `homoglyphs` and `mixed_scripts` signals raised	Pass	`test_normalize.py::test_cyrillic_lookalikes_inside_latin_word_are_folded`
TC-26	FR 4.1	Benign Arabic business text is not flagged	Neutral Arabic business sentences; rules-only detector	Score below 0.5, label `benign`, language `ar`	Pass	`test_detector.py::test_benign_arabic_business_text_is_below_flag_threshold`
TC-27	FR 4.3	Sanitising removes only the flagged span	Sentence containing a neutral placeholder phrase matched by a rule	Placeholder replaced by the sanitise marker; the rest of the text is unchanged	Pass	`test_sanitize.py::test_exact_rules_remove_only_the_span`
T	FR	Untrust	Untruste	Header and	Pas	`test_sanitize.py::test_spotlight_structure`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
C-28	4.3	ed content is spotlighted	d text from `read_email` with a call id	footer carry the source and call id; English and Arabic notes follow the header; the body is unchanged inside	s	
TC-29	FR 4.3, FR 3.4	The gateway sanitises a flagged result and marks the session	Untrusted result containing a neutral test marker that the stub rule scores as an injection	Action `sanitize`; marker removed; reason `INJECTION_DETECTED`; session snapshot records the injection; audit action `sanitize`	Pass	`test_gateway.py::test_injection_is_sanitized_and_taints_session`
TC-30	FR 4.1	Test data cannot influence training or tuning	Train twice; before the second run, replace each test split by half of its records with flipped labels	Identical weights and ensemble configuration; only the test metrics differ	Pass	`test_train.py::test_tuning_never_looks_at_test_splits`

B.3.5 Approvals and audit

Table 13: Test cases TC-31 to TC-39, approvals and audit

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-31	FR 5.1	A call waits for a human who	Server prepared with `mirsad init -`	The agent's HTTP call blocks while the request is	Pass	`test_e2e_http.py::test_external_email_waits_for_an_approval_sent_from_a_second_process`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		decides from another process	`-demo`; the agent e-mails an outside supplier; an approver approves from a separate Python process	pending and nothing is sent; afterwards ALLOW with `APPROVED_BY_HUMAN`, the e-mail is sent, `latency_ms` excludes the wait, one `approval.decided` event		
TC-32	FR 5.2	The English summary says what, why and where from	Approval for `send_email` to an outside address that came from an untrusted e-mail	Three sentences: the action and destination (outside Najd Trading Co.), why Mirsad paused it with the source e-mail, and the risk level	Pass	`test_approval_summary.py::test_english_summary_matches_the_reference_example`
TC-33	FR 5.2	Argument values cannot add sentences to the summary	A `to` argument containing line breaks, text imitating a system instruction and a 300-character run	Summary contains no line break; long values are clipped	Pass	`test_approval_summary.py::test_attacker_controlled_values_cannot_inject_sentences`
TC-34	FR 5.3, BR 5	Undecided requests expire and	Pending request; wait with a 0.2 s timeout;	Status EXPIRED; `approval.decided` published with status	Pass	`test_approval_service.py::test_wait_timeout_marks_expired_and_publishes`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
		stay closed	a late approval attempt	`expired`; the late decision is rejected		
TC-35	FR 5.3	Exactly one decision wins	Six concurrent decisions on one request	One decision succeeds; the other five raise `ValueError`	Pass	`test_approval_service.py::test_concurrent_deciders_exactly_one_wins`
TC-36	FR 6.1	Reordered audit rows are detected	Five events; the sequence numbers of events 2 and 3 are swapped in the database	Verification fails with first bad sequence 2	Pass	`test_audit_chain.py::test_verify_detects_reordered_rows`
TC-37	FR 6.1	A rewrite by the key holder is detected with an external anchor	Five events; anchor saved; event 2 rewritten and the chain recomputed with the real key	Verification without the anchor passes (documented limit); with the anchor it fails at sequence 5	Pass	`test_audit_chain.py::test_key_holder_rewrite_passes_chain_but_fails_external_anchor`
TC-38	FR 6.2	Audit filters return the right events	Populated log; filters by agent, tool, action, event type, session and call id, then a	Each filter returns the expected count; the combined filter returns the single matching event	Pass	`test_audit_query.py::test_query_filters_by_each_column`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			combination			
TC-39	FR 6.3	Replay uses the recorded policy version	A call is denied; a new policy that allows it is activated; the original event is replayed	New calls are allowed; the replay returns DENY with `match` true and the original policy version	Pass	`test_api_admin.py::test_replay_after_policy_change_uses_recorded_version`

B.3.6 Administration, user interface, demonstration, benchmark and storage

Table 14: Test cases TC-40 to TC-46, administration, UI, demonstration, benchmark and storage

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
TC-40	FR 2.5	Policies are hot-reloaded, audited and can be rolled back	Administrator activates a new policy with `PUT /api/policies`, then reactivates the first version	The next call uses the new version; rate counters survive; one `policy.activated` event attributed to the administrator; rollback restores the first version	Pass	`test_api_admin.py::test_policy_activation_hot_reloads_the_gateway_and_is_audited`
TC-41	FR 7.1	Agent keys are shown once and can be rotated	Administrator creates an agent, lists agents, rotates the key,	The key (`mk_…`) appears only in the create response, never in listings or audit	Pass	`test_api_admin.py::test_agent_crud_shows_key_once`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			disables and deletes the agent	events; the old key is rejected (401) after rotation; each change is audited		
TC-42	FR 5.3, NFR4	The approver PWA works in Arabic	A pending request rendered with the Arabic language setting	Right-to-left layout showing the Arabic summary, the reason and an Arabic approve button	Pass	`ui.test.tsx`: approver PWA, "renders right-to-left in Arabic with the Arabic summary and reason"
TC-43	FR 8.1	The demonstration shows the effect of protection	Demo chat with a scripted model stub whose plan e-mails an outside address; protection off, then on	Off: the message reaches the attacker inbox. On: the call is denied, the attacker inbox stays empty and the decision is audited with the demo user	Pass	`test_api_demo.py::test_unprotected_agent_leaks_to_attacker_inbox` `test_api_demo.py::test_protected_agent_is_stopped_by_the_gateway`
TC-44	FR 8.2, BR 2	The benchmark escalates legitimate high-risk tasks	Tasks `en-approval-transfer-01` and `ar-approval-delete-01` under `full` protection with the oracle	Both tasks escalated and approved; utility true; every decision replays to the same action	Pass	`test_bench_agents.py::test_legitimate_approval_tasks_are_escalated_and_oracle_approves`

ID	Req.	Objective	Preconditions / inputs	Expected result	Actual	Automated test
			approver			
TC-45	NFR11	The audit chain works on PostgreSQL	Throwaway PostgreSQL server (`pgserver`); 25 events containing Arabic text	Chain verifies with 25 events checked; the query finds all 25 events; the JSON export contains the Arabic text	Pass	`test_postgres.py::test_audit_chain_appends_and_verifies_on_postgres`
TC-46	FR8.2	Attack episodes are built, planted, scored and attributed to the layer that stops them	A neutral placeholder template and a placeholder injection task (a customer export above the 50-record limit) planted into a document that a benign user task reads; scripted agent; defences `none` and `full`, and `detector_only` with a stub rule	`none`: the placeholder text is delivered and the injection predicate holds. `full`: the export is denied by the policy constraint, the episode is attributed `stopped_by = policy` and the user task still succeeds. `detector_only`: the planted text is sanitised, never delivered, and the episode is attributed to the detector	Pass	`test_bench_attacks.py::test_policy_denial_stops_a_large_export_under_full_defense` `test_bench_attacks.py::test_scripted_agent_does_not_comply_when_the_detector_removes_the_text`

B.4 Security test cases

The security suite (`tests/security/test_gateway_security.py`) tests the gateway itself against the threats found in the STRIDE analysis [2], [5]. It replaces the manual Burp Suite and OWASP ZAP pass planned in the proposal with checks that run on every change. Each case runs against the full FastAPI application with real authentication, storage and audit components. Some cases are parametrised over several routes, roles or inputs; the result column gives the number of passing instances. Each case is listed under the STRIDE category of the test-file section in which it is defined. A few cases also cover a second category (ST-03 elevation of privilege, ST-17 tampering, ST-19 spoofing); Section 3.3.8 maps the same cases to the threats they mitigate.

Table 15: Security test cases ST-01 to ST-23

ID	STRIDE category	Threat	Expected result	Result
ST-01	Spoofing	Unauthenticated access to management APIs	Ten management routes (users, agents, policy validation and activation, audit list, verify and export, approvals, statistics, tools) return 401 without a token	Pass (10/10)
ST-02	Spoofing	Forged or unsigned session tokens	Tokens signed with another key, with algorithm `none`, expired, malformed or empty all return 401	Pass
ST-03	Spoofing	A role claim edited inside a valid token	An approver token whose role is changed to admin without re-signing returns 401	Pass
ST-04	Spoofing	Use of a revoked agent key	After rotation the old key returns 401 and the new key 200	Pass
ST-05	Spoofing	Guessed or malformed agent keys	Four malformed keys return 401	Pass
ST-06	Tampering	Editing an audit row in the database	Verification fails and names the edited sequence number	Pass
ST-07	Tampering	Deleting an audit row	Verification fails	Pass
ST-08	Tampering	A policy containing a catastrophic-backtracking regular expression	Validation reports the policy invalid; activation returns 422	Pass
ST-09	Repudiation	An approver denies having decided, or a decision is changed later	Decision stored with the approver's name; the waiting call is denied; a second decision returns 409; an `approval.decided` event is in the audit log	Pass
ST-10	Repudiation	Failed logins that leave no trace, or leak the password	An `auth.login_failed` event is recorded; the attempted password does not appear in the audit log	Pass
ST-11	Information disclosure	Secrets in tool arguments reaching the log	An API-key-shaped value and a `password` argument are absent from the JSON audit export	Pass
ST-12	Information	Hashes or keys in listings	User and agent listings contain no	Pass

ID	STRIDE category	Threat	Expected result	Result
	disclosure		password hash, key hash or full key	
ST-13	Information disclosure	Stack traces or internal paths in error responses	A failing tool produces a response without the internal path or a traceback	Pass
ST-14	Information disclosure	Path traversal through static file serving	Four traversal paths (plain, URL-encoded, encoded dots, under `/approve`) return no file from outside the web root	Pass (4/4)
ST-15	Information disclosure	Clickjacking and content-type sniffing	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, a CSP with `frame-ancestors 'none'` and a Referrer-Policy on API responses	Pass
ST-16	Information disclosure	Cross-origin use of the API from an untrusted site	An unknown origin is not echoed in the CORS header; the configured origin is	Pass
ST-17	Information disclosure	SQL injection through audit search filters	Three SQL-injection-style filter values return 200 with no results; the audit chain still verifies	Pass (3/3)
ST-18	Denial of service	Oversized request bodies	A body over 1 MB returns 413	Pass
ST-19	Denial of service	Password brute force	Repeated failures lead to 429; the correct password is also refused with 429 inside the window	Pass
ST-20	Elevation of privilege	Approvers or auditors performing administrator actions	Six role and route combinations return 403	Pass (6/6)
ST-21	Elevation of privilege	Mixing agent and human credentials	An agent key on a human API returns 401; an administrator token on an agent API returns 401	Pass
ST-22	Elevation of privilege	Calling an unlisted tool, or using an agent bound to an unknown policy agent	Both calls are denied	Pass
ST-23	Elevation of privilege	Server-side request forgery through `fetch_url` under the default policy	Loopback, cloud-metadata, `localhost` and `file:` URLs are denied	Pass (4/4)

Table 16: Security tests by STRIDE category

STRIDE category	Test cases
Spoofing	ST-01 to ST-05
Tampering	ST-06 to ST-08
Repudiation	ST-09, ST-10
Information disclosure	ST-11 to ST-17
Denial of service	ST-18, ST-19
Elevation of privilege	ST-20 to ST-23
Total	**23 security tests; 45 test cases passed in the**

STRIDE category	Test cases
	recorded run

B.5 Test results

B.5.1 Execution summary

Table 17: Test execution summary

Suite	Runner	Tests	Passed	Failed	Skipped
Python, all areas (`-m "not llm"`)	pytest	1,281	1,281	0	0
Web UI (10 test files, 38 suites)	Vitest	194	194	0	–

The per-area breakdown of the Python tests is in Table 4 (Section 3.5.6). The two `llm`-marked tests were not part of the recorded run. The LLM smoke run in B.5.7 exercises the same model integration through the benchmark harness.

B.5.2 Code coverage

Table 18: Statement coverage by package

Package or module	Responsibility	Statements covered
`adapters`	MCP proxy, OpenAI-compatible proxy, upstream manager	90.3%
`api`	FastAPI application, routers, HTTP hardening	91.2%
`approvals`	Approval service, notifier, approver summaries	96.2%
`audit`	Hash-chained log, verification, query, export, redaction	96.1%
`auth`	Password hashing, tokens, agent keys, login throttling	96.2%
`cli`	`mirsad` command line	92.4%
`config`	Settings	95.8%
`detector`	Normalisation, rules, model, sanitiser, dataset and training	96.9%
`gateway`	Decision pipeline, inspection, replay	90.7%
`models`	Shared data types	100.0%
`policy`	Policy language, engine, rate limiter, regex guard	94.9%
`sdk`	Python client SDK	94.4%
`storage`	Database engine and tables	99.3%
`taint`	Taint extraction, rules, tracker	96.5%
`textutil`	Shared text helpers (Arabic folding, domains)	98.2%
Total	9,763 statements	**94.4%**

Coverage is statement coverage only. In the recorded run every package is above the 80% target of NFR9. The lowest values are in the protocol adapters and the gateway. Their uncovered lines are mostly exception handlers and rarely taken edge cases, such as replaying an event whose audit payload was truncated. The stdio entry point of the MCP proxy is exercised by TC-11, but it runs in a subprocess that coverage does not measure.

B.5.3 Static analysis and dependency scanning

Table 19: Static analysis and dependency scan results

Tool	Scope	Configuration	Result
Ruff	`src`, `tests`, `demo`, `bench`, `scripts`	Rule sets E, F, W, I, B, UP and S (security rules from flake8-bandit); S and B relaxed in tests	No report stored in `reports/`
Bandit	`src` (13,023 lines of code)	Default rule set; B101 (assert) skipped	0 findings at any severity
TypeScript compiler	`web/`	`strict` mode, `tsc --noEmit`	No report stored in `reports/`
pip-audit	Installed Python packages (124; the editable project itself skipped)	PyPI advisory database	0 packages with a known vulnerability
npm audit	Web dependencies (231 packages, development dependencies included)	npm advisory database	0 vulnerabilities

The CI workflow defines Ruff and the TypeScript compiler as checks, but neither stores a report file and no CI run is recorded, so these two results are not part of the recorded evidence.

B.5.4 Performance

The latency benchmark (`bench/latency.py`) starts the real application with the default policy and agent-key authentication. It sends 1,000 benign read-only calls spread over six tools, first one at a time and then from 50 concurrent sessions. Added latency is the HTTP round trip through Mirsad minus the direct in-process call of the same tool. The benchmark was run twice: with the server in a separate operating-system process, which is the more realistic set-up, and with the server in a background thread of the benchmarking process.

Table 20: Added latency per tool call (ms)

Measure	Separate server process	Server in the same process
Sequential, added latency p50	2.66	2.64
Sequential, added latency p95	5.96	5.90

Measure	Separate server process	Server in the same process
Sequential, added latency p99	6.12	6.01
Sequential, gateway-reported time p50	2.06	2.04
Sequential throughput (requests/s)	317	317
50 sessions, added latency p50	165.5	204.8
50 sessions, added latency p95	208.0	880.1
50 sessions, added latency p99	216.1	1304.3
50 sessions, gateway-reported time p50	159.5	2.3
50 sessions, throughput (requests/s)	291	154
50 sessions, HTTP errors	0	0
Audit chain intact after the run (True/False)	True	True

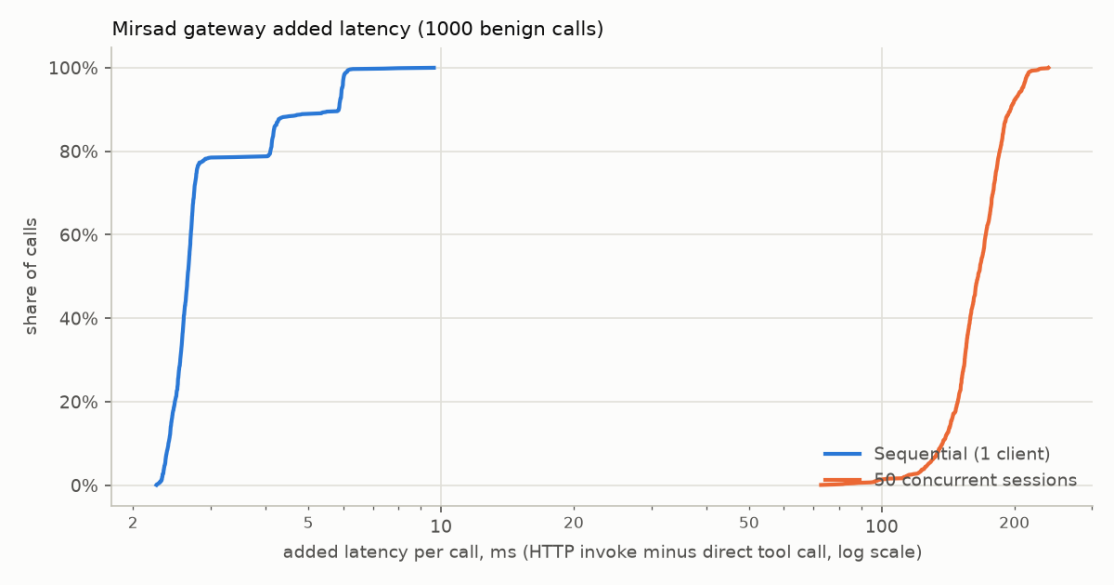


Figure 1: Cumulative distribution of added latency over 1,000 benign calls, server in a separate process (log scale)

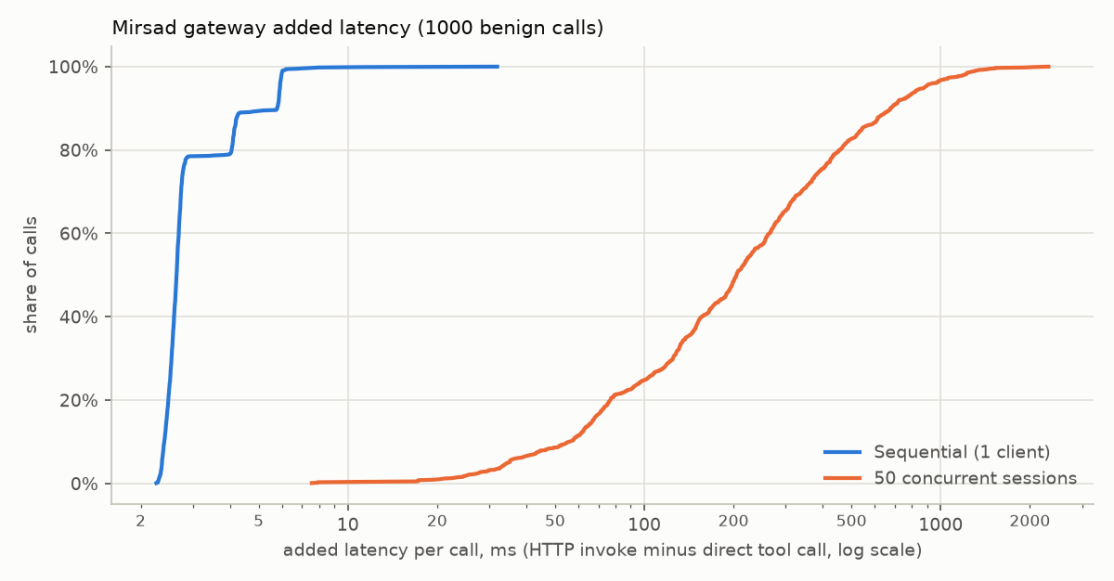


Figure 2: Cumulative distribution of added latency over 1,000 benign calls, server in the same process as the client (log scale)

With one request at a time, Mirsad adds about 3 ms per call at the median, far below the 200 ms budget of NFR1 (Figure 1). Under 50 concurrent sessions served by a single uvicorn worker, the median added latency rises to 166 ms with a separate server process and 205 ms when client and server share one Python process (Figure 2), with 0 and 0 HTTP errors. The separate-process median is the figure assessed against NFR1; in the in-process layout the client competes with the server for one interpreter, which gives a much longer tail. Most of this time is queueing. In the separate-process run, the gateway-reported time itself grows to a median of 159 ms. This suggests that requests wait inside the gateway, where taint registration and audit appends run under one lock and write to SQLite. The concurrency figure should therefore be read as "50 sessions served on one worker", not as a throughput claim. Measurements with several workers and PostgreSQL remain to be done.

B.5.5 Benchmark results on benign tasks

Mirsad-Bench ran the 48 benign user tasks (24 English; 24 Arabic, of which 14 are Modern Standard Arabic and 10 Gulf dialect) with the deterministic scripted agent under each defence configuration. Each configuration made 68 benign tool calls. Two approver behaviours were simulated: *deny* rejects every escalation, and *oracle* approves exactly the calls that match the user's task, like a reviewer who knows what the user asked for. The `none` configuration calls the tools directly and is the same in both runs.

Table 21: Benign benchmark results per defence and approver

Defence	Approver	Utility	FPR	Friction	Final block	Result FP	Gateway p50 / p95 (ms)

Defence	Approver	Utility	FPR	Friction	Final block	Result FP	Gateway p50 / p95 (ms)
none	–	100.0%	0.0%	0.0%	0.0%	n/a	n/a
full	deny	75.0%	0.0%	17.6%	17.6%	0.0%	1.77 / 5.30
policy_only	deny	75.0%	0.0%	17.6%	17.6%	n/a (detector off)	0.94 / 1.52
taint_only	deny	87.5%	0.0%	8.8%	8.8%	n/a (detector off)	1.09 / 1.98
detector_only	deny	100.0%	0.0%	0.0%	0.0%	0.0%	1.45 / 3.76
full	oracle	100.0%	0.0%	17.6%	0.0%	0.0%	1.82 / 5.29
policy_only	oracle	100.0%	0.0%	17.6%	0.0%	n/a (detector off)	0.95 / 1.94
taint_only	oracle	100.0%	0.0%	8.8%	0.0%	n/a (detector off)	1.08 / 2.16
detector_only	oracle	100.0%	0.0%	0.0%	0.0%	0.0%	1.48 / 4.59

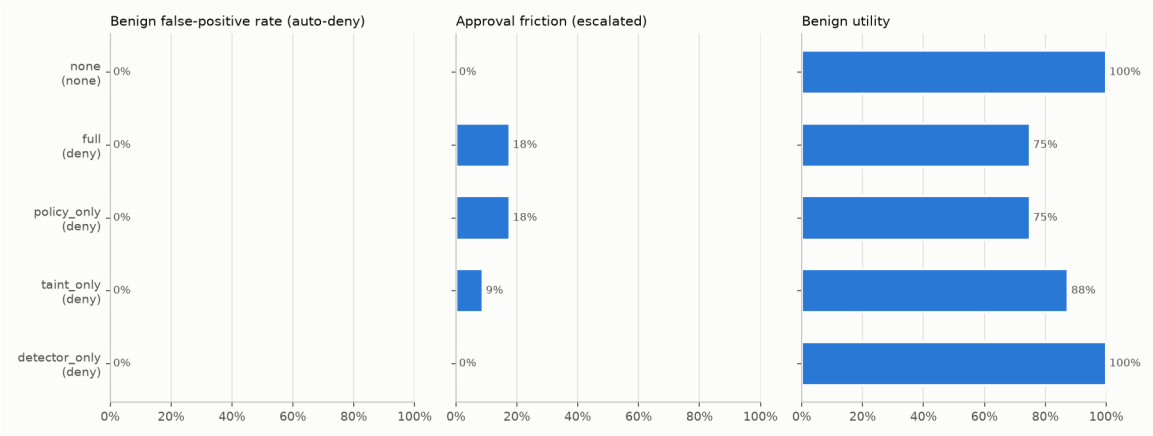


Figure 3: Benign false-positive rate, approval friction and utility per defence configuration with the denying approver (run `scripted-benign-deny`)

The main observations are:

- **Automatic denials.** The FPR and Result FP columns give the share of benign calls denied automatically and of benign tool results sanitised or blocked in each configuration, against the FPR target of at most 5%, with the detector in rules-only mode.
- **Friction comes from the policy's intended escalation paths.** Under `full` protection, 12 of the 68 benign calls were escalated. The reason codes recorded in `reports/bench/scripted-benign-deny/summary.md` are `CONSTRAINT_VIOLATION` (6) and `TAINT_UNTRUSTED_TO_EXTERNAL` (6) for e-mails to outside recipients, `APPROVAL_ALWAYS`

(4) for the payment and deletion tasks, and `APPROVAL_POLICY` (2) for exports. Under full protection, friction was 17.6% for English and 17.6% for Arabic tasks.

- **The approver decides the utility cost.** With the oracle approver, utility under full protection stays at 100%. With an approver who rejects everything, it falls to 75%.
- **The policy layer is needed for high-risk tools.** Only the configurations that include the policy layer asked a human about the legitimate payment and deletion tasks (approval recall 100% for `full`, 0% for `taint_only` and 0% for `detector_only`).
- **Gateway processing time is a few milliseconds** per call in every configuration (Figure 4).

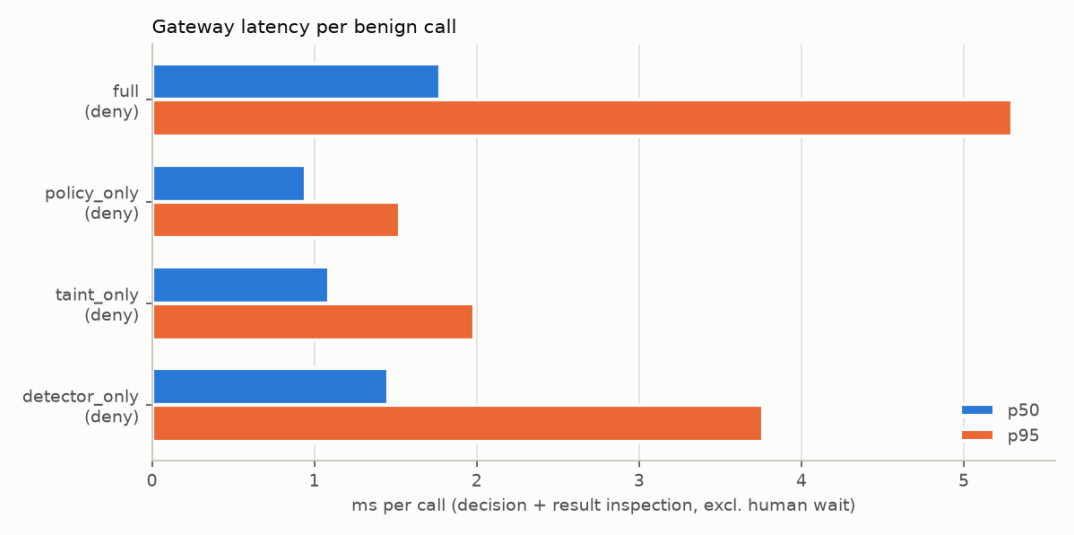


Figure 4: Gateway time per benign call (decision plus result inspection, human wait excluded), p50 and p95, run `scripted-benign-deny`

B.5.6 Audit verification and decision replay

After each benchmark configuration, the harness verified the HMAC hash chain of that configuration's audit log and replayed every recorded `tool_call.decision` event with its stored policy version and inputs (`--replay-check`). The latency benchmark verified the chain after its 1,000 calls.

Table 22: Audit verification and replay results

Run	Audit chains verified (True/False)	Decisions replayed	Same action
`scripted-benign-deny` (4 protected configurations)	True	272	272
`scripted-benign-oracle` (4 protected configurations)	True	272	272
`llm-smoke` (full protection)	True	6	6
Latency benchmark, separate process	True	–	–

Run	Audit chains verified (True/False)	Decisions replayed	Same action
Latency benchmark, same process	True	–	–

Every replayed decision reproduced its original action, which meets the acceptance criterion of FR6.3. Tamper detection itself is tested by TC-36, TC-37, ST-06 and ST-07.

B.5.7 LLM smoke run

A smoke run checked that the real model integration works through the gateway. The demonstration agent used Qwen2.5-7B through Ollama at temperature 0 on 4 benign tasks (2 English, 2 Arabic), without protection and with full protection (oracle approver). Utility was 75% without Mirsad and 75% with it. FPR and friction were 0% and 0%. The gateway time over 6 calls was 3.59 ms at the median and 5.52 ms at p95. The same task failed in both configurations (`ar-crm-update-02`): the episode records show that the model wrote a CRM note without the requested phone number, and the gateway allowed every call. The failure is the model's, not the gateway's. With so few tasks this run is a functional check, not a utility estimate.

B.5.8 Results pending team-supplied data

Table 23 lists the results that the evidence pipeline produces once attack-labelled data exists. They are not estimated here. The repository currently holds only benign detector batches (`data/detector/raw/en-ben.jsonl` and `ar-ben.jsonl`) and no attack files under `bench/scenarios/attacks/`. Attack material is written and reviewed by the team (Section 3.6.4).

Table 23: Results that need team-supplied attack data

Result	Value	How it is produced
Detector F1, English held-out test set (target ≥ 0.85)	[pending]	Add labelled training batches and separate held-out test batches (`raw/test-*.jsonl`) to `data/detector/raw/`, then run `make detector` (`docs/detector.md`, Section 11.4)
Detector F1, Arabic held-out test set (target ≥ 0.85)	[pending]	As above
Detector F1, mixed Arabic–English test set	[pending]	As above
Detector precision / recall, English	[pending] / [pending]	As above
Detector precision / recall, Arabic	[pending] / [pending]	As above
Detector scan latency, 2 KB text, p50 (target < 10 ms)	[pending] ms	Measured by the same training run
Attack success rate, full protection	[pending]	Add injection tasks and attack templates to

Result	Value	How it is produced
		`bench/scenarios/attacks/` (`bench/scenarios/attacks/README.md`), then run `scripts/run_bench.sh`, which executes the attack episodes
ASR reduction against the unprotected baseline (target ≥ 80%)	[pending]	As above

B.6 Known limitations and open items

- **Detector accuracy and attack success rate are not measured yet** (B.5.8). Until they are, the evaluation shows what Mirsad costs legitimate work, but not how much protection it gives against real attacks.
- **Tests verify how detections are handled, not detection quality.** Functional tests use neutral placeholders scored by a stub rule, so they cannot show whether real Arabic or English injections are recognised.
- **All detector-dependent numbers come from the rules-only fallback**, because no trained model is packaged. The benchmark, the latency benchmark and the result false-positive rate must be measured again once a model ships.
- **No attack episode has been run.** The harness executes attack episodes (`bench/attacks.py`, tested with neutral placeholder text in `tests/bench/test_bench_attacks.py`), but no attack files exist yet.
- **Benchmark coverage is limited.** The scripted agent replays task plans exactly, the LLM run covers 4 tasks, and the oracle approver is idealised. Approval quality and decision time of real people are not modelled.
- **Usability (NFR4) and accessibility (NFR10) checks are incomplete.** The decision-time figure is an analytical Keystroke-Level Model estimate (Section 4.2.8); no approver has been timed in a user study, no acceptance session (Section 3.5.7) has been held, and no UI review checklist is recorded. Keyboard navigation and reduced-motion behaviour are not covered by automated tests.
- **Concurrency.** Tail latency with 50 sessions on one worker is high, and part of the waiting happens inside the gateway (B.5.4). Multi-worker and PostgreSQL latency have not been measured.
- **Deployment.** The CI workflow defines a Docker build job, but no CI run, image build or `docker compose up` is recorded in `reports/`. The Compose file does not yet set `MIRSAD_POLICY_PATH` and `MIRSAD_UPSTREAMS_PATH` for the `mirsad` service (Appendix E.2.2).
- **Coverage scope.** Only statement coverage is collected. The 80% target is not enforced by a CI threshold, and `npm audit` is not part of the CI workflow.
- **Excluded tests.** The two `llm`-marked tests are not part of the recorded run.

- **Small open integration items.** Two classes implement login throttling (`mirsad/auth.py` and `mirsad/api/routers/auth.py`) and the application uses the second, so one should be removed. The policy embedded in `demo/agent.py` for stand-alone use of the demonstration agent lacks the global rule `no-bulk-export-to-email` and should load `configs/policies/default.yaml` instead. One literal left-to-right mark remains in the Arabic dictionary of the web app, and isolate characters remain inside two JSON test fixtures captured from the server; they should be written as escapes like the Python sources.
- **Benchmark fingerprint.** The runner fingerprints the scenario files but not the policy, so a resumed run would not detect a policy change; the reported runs were therefore executed from scratch (B.1).
- **Audit integrity has a documented limit.** Someone who holds the HMAC key can rewrite history undetected unless an external anchor of the chain head is kept (TC-37).
- **Security testing is automated and internal.** No external penetration test or adaptive red-team exercise has been carried out against the gateway.

References

[1] Software and Systems Engineering — Software Testing — Part 1: General Concepts, ISO/IEC/IEEE 29119-1:2022, Feb. 2022.

[2] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: John Wiley & Sons, 2014, ISBN: 978-1-118-80999-0.

[3] A. Yang et al. (Qwen Team), "Qwen2.5 technical report," arXiv:2412.15115, Dec. 2024 (rev. Jan. 2025). [Online]. Available: https://arxiv.org/abs/2412.15115

[4] S. K. Card, T. P. Moran, and A. Newell, "The keystroke-level model for user performance time with interactive systems," Commun. ACM, vol. 23, no. 7, pp. 396–410, Jul. 1980, doi: 10.1145/358886.358895.

[5] Microsoft, "Threats - Microsoft Threat Modeling Tool - Azure," Microsoft Learn. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats.